TRUMBO RESEARCH

Quartz

A Private-Weights, Long-Context Foundation Model for Reasoning and Agentic Work

Technical architecture, adaptive inference, tool-grounded execution, and reliability design

Shubhankar Kahali
Kai Keskitalo
Zuzanna Kowalczyk

Trumbo | July 2026

Scope of this paper

This document is a technical system paper. It describes Quartz design targets, operating assumptions, and evaluation methodology while deliberately withholding private weights, parameter counts, training-data composition, and unreleased benchmark results.

Abstract

Quartz is Trumbo's closed-source foundation-model system for long-context understanding, multimodal reasoning, and reliable agentic work. This paper defines the architectural thesis behind Quartz: a private sparse-expert core couples a hybrid long-context sequence pathway with adaptive inference, verification, structured generation, and bounded tool use. The design treats quality as a system property. A capable decoder alone is insufficient when a task requires codebase-scale context, high-consequence reasoning, tool execution, or machine-readable outputs. Quartz therefore organizes inference around five interacting layers: input and context assembly, a long-context model backbone, conditional expert computation, adaptive reasoning and action loops, and a reliability contract at the output boundary.

The paper does not claim a released parameter count or benchmark ranking. Instead, it states the implementation-oriented principles that a frontier private model system should satisfy: preserve evidence across long contexts; apply specialist capacity without turning product behavior into a collection of disconnected systems; expand inference compute in proportion to uncertainty; bind tool actions to explicit schemas and permissions; and emit answers that can be validated by both machines and humans. The resulting architecture is designed to support direct chat, software engineering, document intelligence, structured automation, and long-running agent workflows under one coherent Quartz identity.

Keywords

foundation models; sparse experts; long-context modeling; adaptive inference; process verification; tool use; structured generation; multimodal reasoning; private weights

Reader's Guide

Sections 1-3 establish the model-system thesis and its major components. Sections 4-8 describe the sequence backbone, sparse expert core, adaptive reasoning, agentic execution, and output contract. Sections 9-12 cover training, evaluation, safety, and limitations. The diagrams are logical architecture views: they are intended to explain responsibility boundaries and information flow, not disclose proprietary topology or scale hyperparameters.

1. Introduction

A contemporary general-purpose model is expected to do far more than continue text. It must synthesize across files, reason over ambiguous requirements, interpret visual artifacts, invoke tools safely, recover from errors, and return results in formats that downstream systems can consume. These demands expose a gap between a single-pass language-model interaction and a durable intelligent system. Quartz addresses that gap by treating the model, its memory behavior, its reasoning budget, its tool interface, and its verification procedures as one integrated architecture.

The Quartz thesis is intentionally model-first. Its public surface is a family of modes sharing one private-weights identity and a common behavioral contract. Inside that system, the model uses conditional computation and adaptive inference to allocate capacity where it matters. The purpose is not to fragment an interaction into unrelated services; it is to give one coherent model substrate the ability to use distinct internal circuits, memory paths, and inference procedures for distinct kinds of work.

Design principle

Quartz should remain legible as one model system: the user supplies an objective, the system forms and updates an internal working state, and the final response carries a single coherent identity, safety posture, and reliability contract.

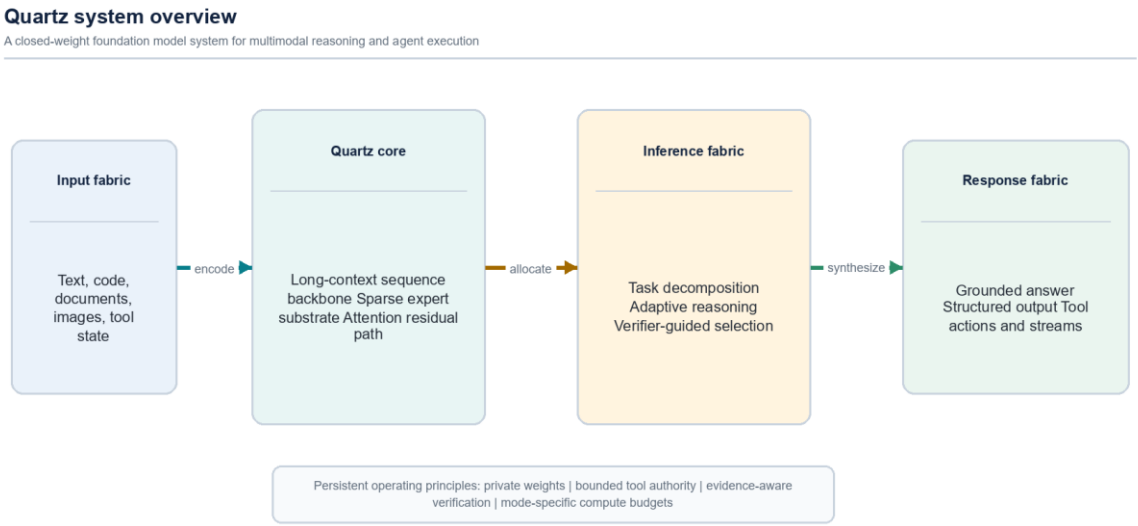

Figure 1. *Quartz system overview*

2. Design Objectives

Quartz is organized around six design objectives. First, it must preserve useful evidence across very long inputs without making every token interaction equally expensive. Second, it must expose a large private capacity while keeping per-request computation bounded through conditional expert activation. Third, it must vary test-time effort with task difficulty, ambiguity, and consequence. Fourth, it must work natively with tools and structured interfaces rather than treating them as prompt-only conventions. Fifth, it must

support multimodal evidence as part of the same reasoning workspace. Sixth, it must make reliability observable through verification, traceable tool results, schema conformance, and evaluation gates.

Objective	Design response	Observable evidence
Long-context fidelity	Hybrid sequence pathways, stable-prefix reuse, segment attribution	Retrieval fidelity and multi-document synthesis tests
High capacity at bounded cost	Sparse internal expert activation with shared pathways	Load balance, expert utilization, quality-cost curves
Adaptive reasoning	Difficulty estimation and verifier-guided expansion	Calibration, improvement per extra inference step
Agent reliability	Schema-bound tools, permission boundaries, observation loops	Tool success, recovery, and unsafe-action refusal tests
Machine usability	Strict structured decoding and continuation support	Schema validity, parsing success, deterministic contract tests

3. Architectural Overview

Quartz can be read as five layers arranged along a single inference path. The input fabric turns text, code, documents, images, and tool observations into typed model context. The core model fabric transforms that context through a long-sequence backbone and a sparse expert substrate. The inference fabric decides how much internal work a request merits and whether it requires a tool-grounded loop. The verification fabric checks intermediate and final candidates against task constraints. The response fabric emits natural language, structured data, actions, and streams under one output contract.

This decomposition is not merely diagrammatic. It makes technical responsibilities explicit. The context layer owns provenance and stable reuse. The backbone owns representation learning. The expert layer owns conditional specialization. The inference layer owns deliberation depth. The tool layer owns executable interaction. The output layer owns schema, presentation, and transport. By separating responsibilities while retaining a shared model state, Quartz avoids the common failure mode in which agent behavior becomes an opaque collection of prompt templates.

4. Long-Context Representation

4.1 Hybrid Sequence Pathway

Long-context quality is not equivalent to accepting a large number of tokens. A model must preserve local details, carry information over distant spans, and recover source-specific evidence when it is needed. Full attention offers rich token-to-token interaction but has unfavorable growth as sequence length rises. Linear or recurrent-style pathways scale more gently, but can lose the expressive precision required for local code, tables, or cross-reference resolution. Quartz therefore uses a hybrid sequence design: a

selective attention path handles high-information neighborhoods and retrieval-sensitive interactions, while a gated linear memory path carries a compact state over extended spans. A learned fusion mechanism combines both paths at each block.

The practical goal is not to choose one universal attention mechanism. It is to recognize that model context contains different information geometries. A source citation, a function signature, and a table header often require sharp local alignment. A large document history, a user preference, or an evolving plan may benefit more from stable state transport. The hybrid pathway allows Quartz to reserve high-resolution attention for the first class of signals and a more economical memory process for the second. Gated linear attention research provides a useful antecedent: linear-time recurrent formulations can be competitive when gating and hardware-aware execution recover much of the expressiveness lost by simpler linear mechanisms [2].

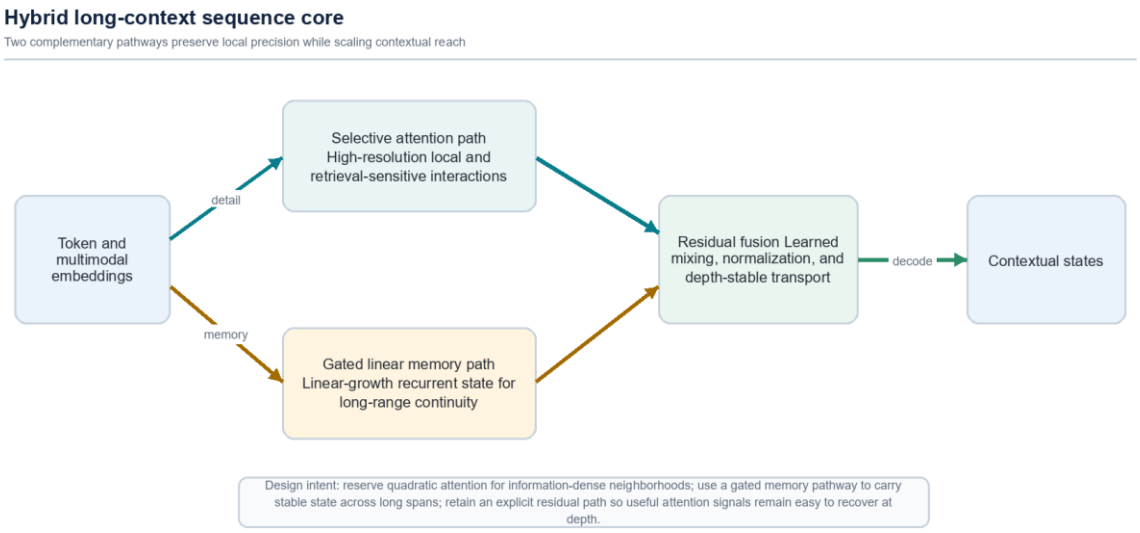

Figure 2. *Hybrid long-context sequence core*

4.2 Attention Residual Transport

Deep sequence models can lose useful attention-derived information as hidden states pass through many transformations. Quartz uses explicit residual transport around attention-derived features as a design requirement. The residual path is not a substitute for learning; it is a controlled route through which salient token relationships can remain recoverable. The mixer learns when to preserve, amplify, or attenuate that signal. This improves the model's ability to reconcile recent precision with longer-running state, especially in contexts that mix short critical spans with large reference material.

4.3 Context Assembly, Caching, and Provenance

Context is assembled as a structured object, not as an undifferentiated prompt string. Stable prefixes such as system instructions, repository indexes, or recurring knowledge packs can be keyed and reused when unchanged. Mutable segments such as a new user turn, a diff, or a tool result are composed around that stable state. A segment index records source identity, position, and summary anchors. This permits

the model to revisit relevant spans and allows the verification layer to distinguish model inference from tool-observed evidence.

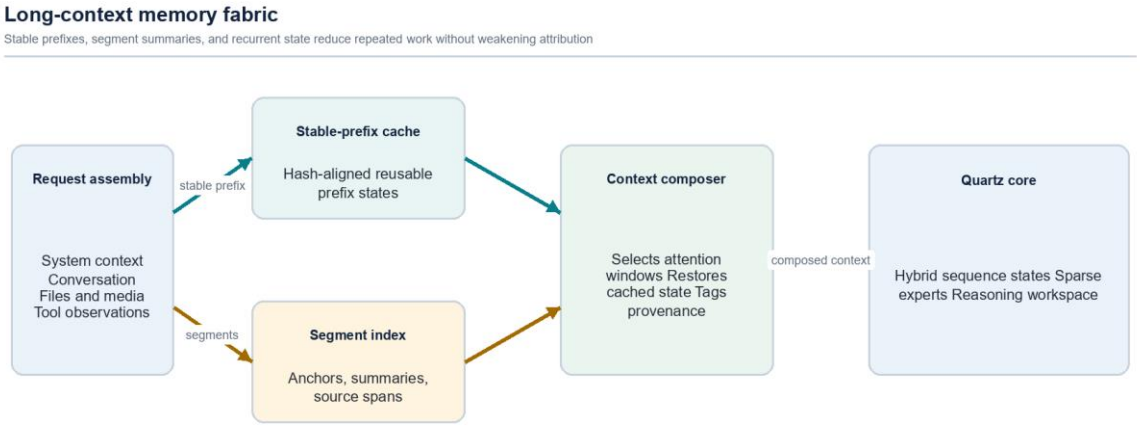

Figure 3. *Long-context memory fabric*

5. Private Sparse Expert Core

Quartz exposes a single private-weights model identity while internally using sparse, conditionally activated expert circuits. Mixture-of-experts research demonstrates that conditional activation can increase model capacity without requiring every input to traverse every parameter group [3]. For Quartz, the important point is architectural coherence: expert selection occurs inside the model substrate at token or span granularity. It is an internal computation decision that is fused back into shared hidden states, not a change in external identity or a handoff between unrelated products.

The sparse-expert block begins with shared normalized hidden states. A learned selector computes affinity scores over a large internal expert inventory and selects a small active set under capacity constraints. Selected experts may specialize along emergent dimensions such as formal reasoning, code transformation, multilingual synthesis, planning, or tool-state interpretation. The selector is trained with load-balance and stability objectives so that it does not collapse onto a small number of popular experts. Expert outputs are weighted, merged, and passed through residual pathways that preserve general instruction-following behavior.

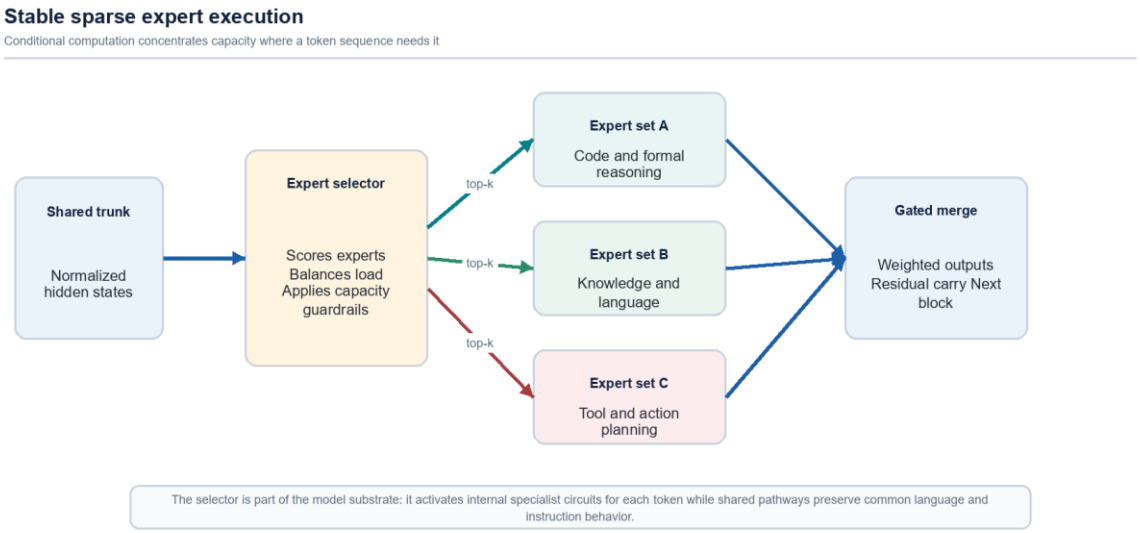

Figure 4. *Stable sparse expert execution*

5.1 Stability Requirements

Conditional capacity adds failure modes that dense models do not have. Expert load can become imbalanced; routing can become brittle under distribution shift; and an expert can over-specialize to superficial cues. Quartz therefore treats three controls as first-class: auxiliary load regularization, route diversity monitoring, and capacity-aware fallbacks. During training, utilization statistics and cross-domain validation detect collapse early. During inference, the shared trunk and residual pathways ensure that a degraded expert choice does not erase a model-wide representation. This is particularly important for long-context and agentic tasks, where token distributions can shift sharply as tool observations enter the conversation.

6. Adaptive Inference and Test-Time Reasoning

A single fixed inference budget is poorly matched to heterogeneous work. A short transformation with explicit instructions should be fast. A mathematical proof, a codebase change, or a high-consequence structured action may merit additional planning, candidate generation, checking, and revision. Quartz implements adaptive inference as a continuous control problem. A difficulty estimator reads task features such as scope, ambiguity, number of constraints, context complexity, prior failure signals, and required tool interaction. It then selects a reasoning policy and a bounded compute budget.

The direct-response policy is appropriate when the model has high confidence and the result is easy to validate. The reasoning policy creates a working plan, derives candidate solutions, and applies focused critique. The tool-grounded policy adds environment interaction: the model proposes schema-valid actions, observes results, updates state, and either continues or terminates. Across all policies, a verifier ranks or rejects candidates based on task-specific criteria. Process supervision is a useful reference point here: step-level feedback can improve reliability on multi-step reasoning tasks because it identifies an error before it reaches a plausible-looking final answer [4].

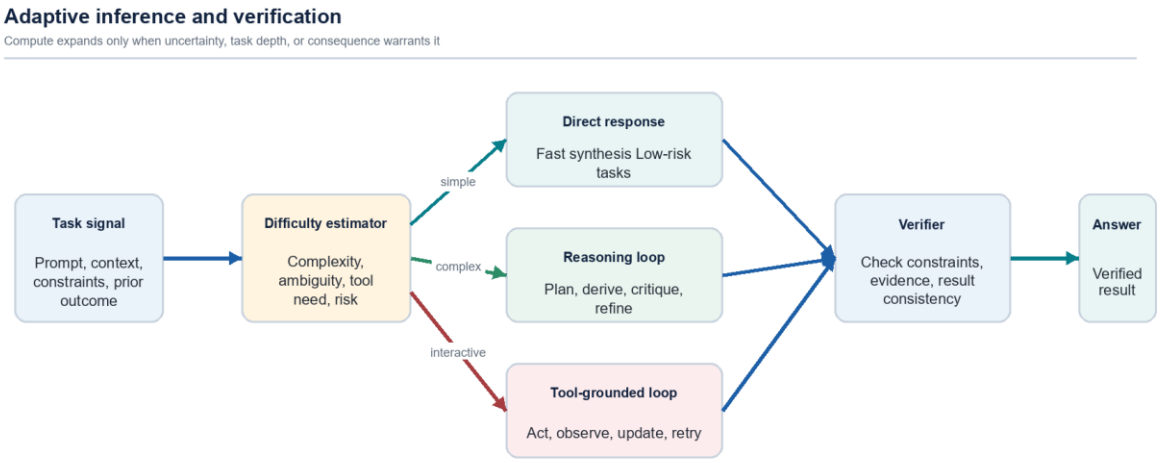

Figure 5. *Adaptive inference and verification*

6.1 Verifier-Guided Candidate Selection

Verification should be discriminating rather than ceremonial. Quartz verifiers operate at multiple granularities. A syntax verifier can test whether a structured answer conforms to a schema. A code verifier can run compilation or tests in an authorized environment. A factuality verifier can check whether cited evidence is present in the assembled context or returned by a tool. A reasoning verifier can inspect whether a candidate violates explicit constraints or contains internal inconsistencies. The verifier does not need to recreate the answer; it needs a narrow, auditable criterion that separates acceptable and unacceptable candidates. This lets Quartz spend additional compute where a detectable uncertainty remains.

7. Tool-Grounded Agentic Execution

Tool use turns language-model inference into stateful control. That creates power and risk at the same time. Quartz therefore treats tool calls as typed transitions, not prose suggestions. A tool declaration specifies a name, purpose, JSON schema, permission scope, side-effect class, and result schema. The model may only emit calls that satisfy the declared structure. An execution policy then determines whether the call is allowed in the current task and whether it requires confirmation, sandboxing, or a more restrictive mode.

The core loop is goal, plan, act, observe, verify, and synthesize. Reasoning traces formulate a plan and track progress; actions retrieve or transform external state; observations become typed context; and verification checks whether the goal has advanced or a recovery path is required. This approach follows the broader lesson from reasoning-and-acting research: external actions can reduce hallucination and enable adaptive plans when their results are integrated back into the model state [5]. Quartz adds bounded authority and explicit stopping rules so that an agent does not continue simply because it can.

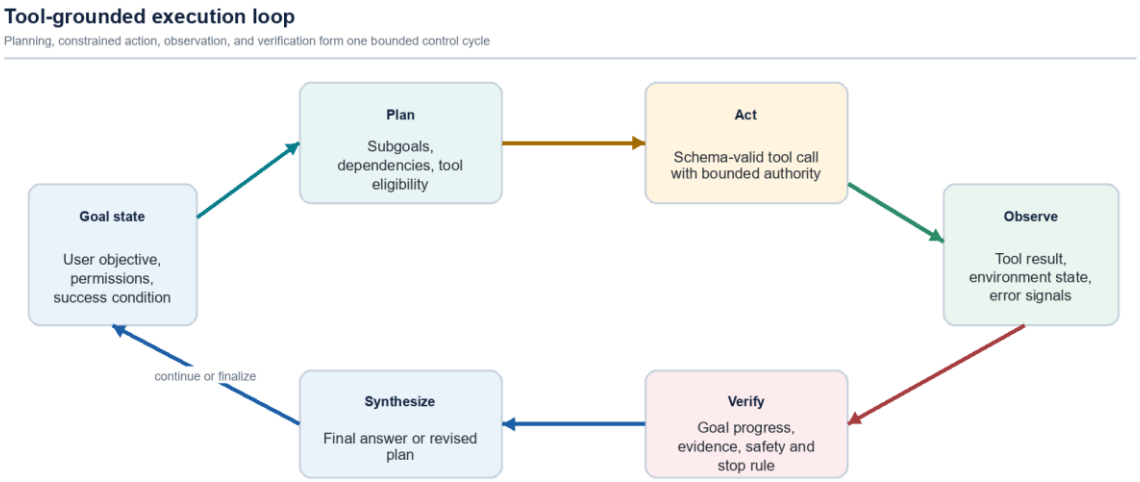

Figure 6. *Tool-grounded execution loop*

7.1 Dynamic Capability Loading

Agent tasks do not need every tool exposed at every moment. Overexposure increases prompt size, tool-selection ambiguity, and attack surface. Quartz supports dynamic capability loading: a planner may request a relevant tool definition when the task reaches a subgoal that needs it. The definition remains part of the model context for the duration of the applicable workflow, so the model can reason over its complete schema and examples. This keeps the operational surface smaller while preserving composability.

7.2 Deterministic and External Computation

Some classes of work benefit from deterministic computation rather than additional natural-language inference. Arithmetic, parsing, code execution, retrieval, and file operations should be delegated to authorized deterministic tools when the task calls for them. Quartz uses the model to decide what to compute, explain the result, and integrate observed outputs. The tool, not the model, produces the authoritative external result. This division makes failures easier to diagnose and makes the final response more grounded.

8. Output Reliability and Developer Interface

The last stage of generation should not be a mere text stream. Quartz supports a response contract that distinguishes internal working state from user-visible content. The output decoder may target natural language, strict structured data, tool calls, partial continuations, or multi-channel streams. For structured tasks, decoding is constrained by an explicit JSON Schema or equivalent grammar. The decoder masks invalid continuations and the verifier checks the final object before it is emitted. Structured-generation research shows why this matters: schema compliance is measurable, and real-world schemas exercise a broader range of constraints than toy examples [6].

Partial continuation is useful when a developer needs the model to complete a preexisting prefix, a document scaffold, or an incremental code stream. The prefix is treated as a deliberate part of the response contract, not as a hidden prompt transformation. Streaming also benefits from separation: progress or reasoning-status events can be transported distinctly from final user-facing content and schema-bound artifacts. The final response remains the authoritative result; intermediate progress must not be mistaken for a verified conclusion.

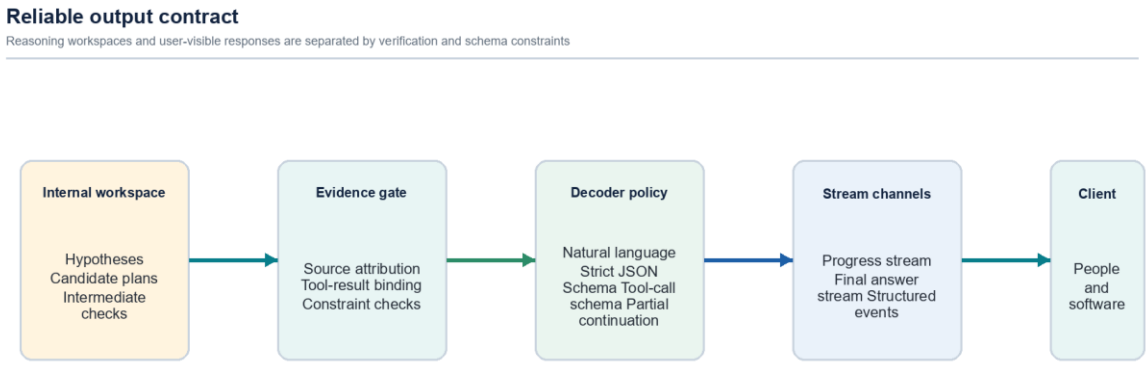

Figure 7. *Reliable output contract*

9. Training and Alignment Program

A credible private model system requires a training program that aligns architectural choices with data, objectives, and evaluation. Quartz training begins with data curation: deduplication, quality filtering, document-structure preservation, code validity checks, multimodal alignment, contamination control, and domain balancing. Foundation training then emphasizes sequence stability, long-context curricula, sparse-expert utilization, and robust instruction representation. Capability shaping adds task families that exercise reasoning, code generation and repair, tool calling, structured output, and multimodal synthesis.

Post-training should combine preference data with process-aware signals. Preference learning helps calibrate helpfulness, style, and policy adherence. Process signals help identify where a multi-step trajectory deviates from a valid path. Tool tasks should include observation noise, API errors, stale state, and permission failures rather than only clean happy paths. Alignment is complete only when it is paired with adversarial evaluation: prompt injection, deceptive tool descriptions, sensitive-data requests, unsafe action chains, and long-context distraction attacks are all relevant to a system that can act.

Figure 8. *Training and alignment lifecycle*

10. Evaluation Methodology

Quartz evaluation is a release discipline, not a one-time leaderboard event. The benchmark program should use versioned task suites, hidden holdouts, controlled ablations, and failure-trace review. Scores must be joined with compute, latency, schema validity, tool success, and safety measures. A model that improves a headline reasoning score while silently increasing unsafe action rates or degrading long-context attribution has not improved in the way Quartz is designed to improve.

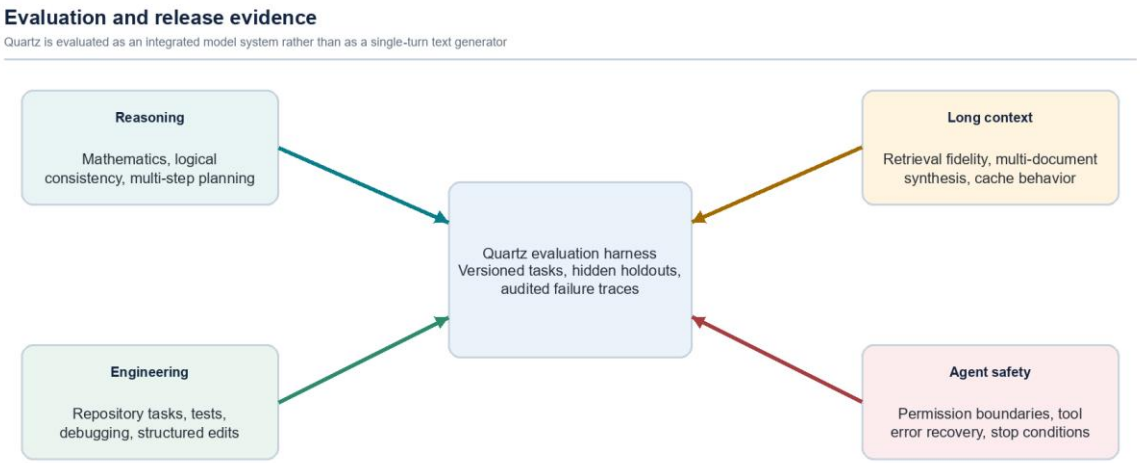

Figure 9. *Evaluation and release evidence*

Area	Representative task	Primary measure	Failure evidence
Reasoning	Constraint satisfaction, proofs, planning	Accuracy with calibrated confidence	Invalid step, contradiction, unsupported conclusion

Area	Representative task	Primary measure	Failure evidence
Software engineering	Issue resolution in real repositories	Test pass rate and patch validity	Regression, incomplete edit, fabricated file state
Long context	Multi-document retrieval and synthesis	Evidence recall, attribution, distractor resistance	Lost source, wrong binding, stale cache use
Structured output	Nested schema extraction and generation	Parse rate and schema conformance	Invalid object, omitted requirement, extra field
Tools and agents	Multi-step environment tasks	Goal completion under bounded authority	Unsafe call, loop, ignored observation, bad recovery
Multimodal	Document, screenshot, and visual reasoning	Grounded answer quality	Visual hallucination, missed region, unsupported claim

10.1 No Unreleased Claims

This paper intentionally does not publish parameter counts, training-token counts, benchmark ranks, or comparisons to other named systems. Those claims require a separate report with fixed model versions, disclosed evaluation protocol, confidence intervals where appropriate, and reproducible artifacts or clearly stated access limitations. The purpose here is to make Quartz's design logic inspectable before a performance report is issued.

11. Security, Privacy, and Safety

Quartz is designed for private weights and controlled operations. That architecture does not itself guarantee security; the deployment must enforce it. Input isolation, tenant boundaries, access logging, secrets handling, data retention rules, and tool credentials are deployment responsibilities. At the model-system layer, Quartz contributes typed context, provenance tags, declared tool permissions, risk-aware inference policies, and stop rules. These controls make safety behavior observable and testable rather than relying on an instruction string to carry the full burden.

Risk	Quartz control	Verification expectation
Prompt injection in external content	Trust labels, source-aware context, restricted tool activation	Injected instructions cannot broaden tool authority
Unsafe or irreversible actions	Permission classes, confirmation gates, sandbox defaults	High-impact calls are blocked or escalated
Hallucinated external state	Tool-result binding and provenance checks	Claims about tools cite returned observations
Long-context distraction	Segment anchors, targeted retrieval, evidence verification	Critical constraints survive distractor-heavy inputs
Structured interface drift	Strict schemas and contract tests	Invalid response objects do not

Risk	Quartz control	Verification expectation
		reach clients

12. Limitations and Open Research Questions

The architecture described here is ambitious and has real tradeoffs. Hybrid sequence paths introduce implementation complexity and require careful calibration of which interactions deserve high-resolution attention. Sparse experts improve conditional capacity but create routing stability and systems challenges. Adaptive inference can increase reliability, yet poorly calibrated expansion wastes compute or creates a false sense of certainty. Tool use grounds some tasks while increasing the need for environment isolation and permission design. Verification is only as good as the criterion it can test; open-ended questions may remain difficult to score without overfitting to superficial form.

Several questions remain open. How should long-context cache reuse be validated when source documents change subtly? How can expert routing remain interpretable without revealing sensitive internal weights? When should a verifier trigger another reasoning pass instead of returning uncertainty to the user? How should a multimodal model preserve exact visual grounding across long videos or dense technical diagrams? And how can evaluation distinguish genuine agentic competence from benchmark-specific tool patterns? Quartz treats these as research questions, not solved marketing claims.

13. Conclusion

Quartz is designed as a coherent private model system for work that exceeds a single text completion. Its technical thesis combines hybrid long-context representation, stable sparse expert computation, adaptive inference, verifier-guided selection, typed tool execution, and schema-constrained output. These mechanisms are mutually reinforcing: memory makes evidence available; experts provide conditional capacity; adaptive inference spends time where it matters; tools connect the model to authoritative state; verification protects the response boundary; and a unified output contract makes the system usable by people and software alike.

The next legitimate step is measurement. A separate technical report should publish fixed-version evaluation evidence, operational envelopes, and limitations once they are ready for review. Until then, the contribution of this paper is an explicit architecture: what Quartz is intended to be, how its major subsystems fit together, and what a rigorous release standard should demand.

References

[1] A. Vaswani et al. Attention Is All You Need. NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[2] S. Yang et al. Gated Linear Attention Transformers with Hardware-Efficient Training. arXiv:2312.06635, 2023. https://arxiv.org/abs/2312.06635

[3] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR, 2022. https://arxiv.org/abs/2101.03961

[4] H. Lightman et al. Let's Verify Step by Step. arXiv:2305.20050, 2023. https://arxiv.org/abs/2305.20050

[5] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. https://arxiv.org/abs/2210.03629

[6] S. Geng et al. Generating Structured Outputs from Language Models: Benchmark and Studies. arXiv:2501.10868, 2025. https://arxiv.org/abs/2501.10868

[7] W. X. Wang et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508, 2023. https://arxiv.org/abs/2308.14508

[8] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL, 2024. https://arxiv.org/abs/2307.03172

[9] C. B. Mann et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. https://arxiv.org/abs/2310.06770

[10] X. Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. ICLR, 2024. https://arxiv.org/abs/2307.13854

[11] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023. https://arxiv.org/abs/2203.11171

[12] V. Venktesh, M. Rathee, and A. Anand. Trust but Verify! A Survey on Verification Design for Test-time Scaling. arXiv:2508.16665, 2025. https://arxiv.org/abs/2508.16665

Appendix A. Quartz Operating Modes

Quartz modes are operational compute profiles, not separate public identities. They maintain the same private-weights model system, tool contract, and safety policy while differing in the default inference budget and latency envelope. Product-specific eligibility and billing policy are intentionally outside the scope of this technical paper.

Mode	Default posture	Suitable work	Escalation behavior
Quartz Lite	Latency-aware direct synthesis with bounded checking	Chat, extraction, small edits, routine transformations	Escalates only for explicit tool requirements or high uncertainty
Quartz	Balanced reasoning and verification	General knowledge work, coding assistance, document analysis	Uses a reasoning loop when complexity and consequence rise
Quartz Hyper	Maximum permitted deliberation and verification	Long-horizon engineering, complex analysis, tool-grounded workflows	Expands planning, candidates, and verification within safety limits

Appendix B. Diagram Reading Notes

The diagrams are conceptual. Arrows indicate the primary direction of information or control flow. They do not imply a one-to-one implementation module, public endpoint, parameter group, or deployment boundary. The sparse-expert diagram is especially important: it represents internal conditional circuits within one model system. The agent loop likewise shows a bounded control cycle, not unrestricted autonomy.